ECG Signal Classification Using Deep Learning

In [1]:
```
import pandas as pd
import numpy as np

import tensorflow as tf
from tensorflow.keras import optimizers, losses, activations, models, utils
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, LearningRateScheduler, ReduceLROnPlateau
from tensorflow.keras.layers import Dense, Input, Dropout, Conv1D, MaxPool1D, Flatten
from sklearn.metrics import f1_score, accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn import metrics
```

In [2]:
```
train = pd.read_csv("mitbih_train.csv", header=None)
test = pd.read_csv("mitbih_test.csv", header=None)
```

In [3]:
```
train.head()
```

Out[3]:

	0	1	2	3	4	5	6	7	8	9	...	178	179	180	181	182	183	184
0	0.977941	0.926471	0.681373	0.245098	0.154412	0.191176	0.151961	0.085784	0.058824	0.049020	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.960114	0.863248	0.461538	0.196581	0.094017	0.125356	0.099715	0.088319	0.074074	0.082621	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	1.000000	0.659459	0.186486	0.070270	0.070270	0.059459	0.056757	0.043243	0.054054	0.045946	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.925414	0.665746	0.541436	0.276243	0.196133	0.077348	0.071823	0.060773	0.066298	0.058011	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.967136	1.000000	0.830986	0.586854	0.356808	0.248826	0.145540	0.089202	0.117371	0.150235	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 188 columns

In [4]:
```
print(train.shape, test.shape)
```

```
(87554, 188) (21892, 188)
```

In [5]:
```
print("Train Dataset Information:")
print(train.info())
print("\nTest Dataset Information:")
print(test.info())
```

```
Train Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 87554 entries, 0 to 87553
Columns: 188 entries, 0 to 187
dtypes: float64(188)
memory usage: 125.6 MB
None

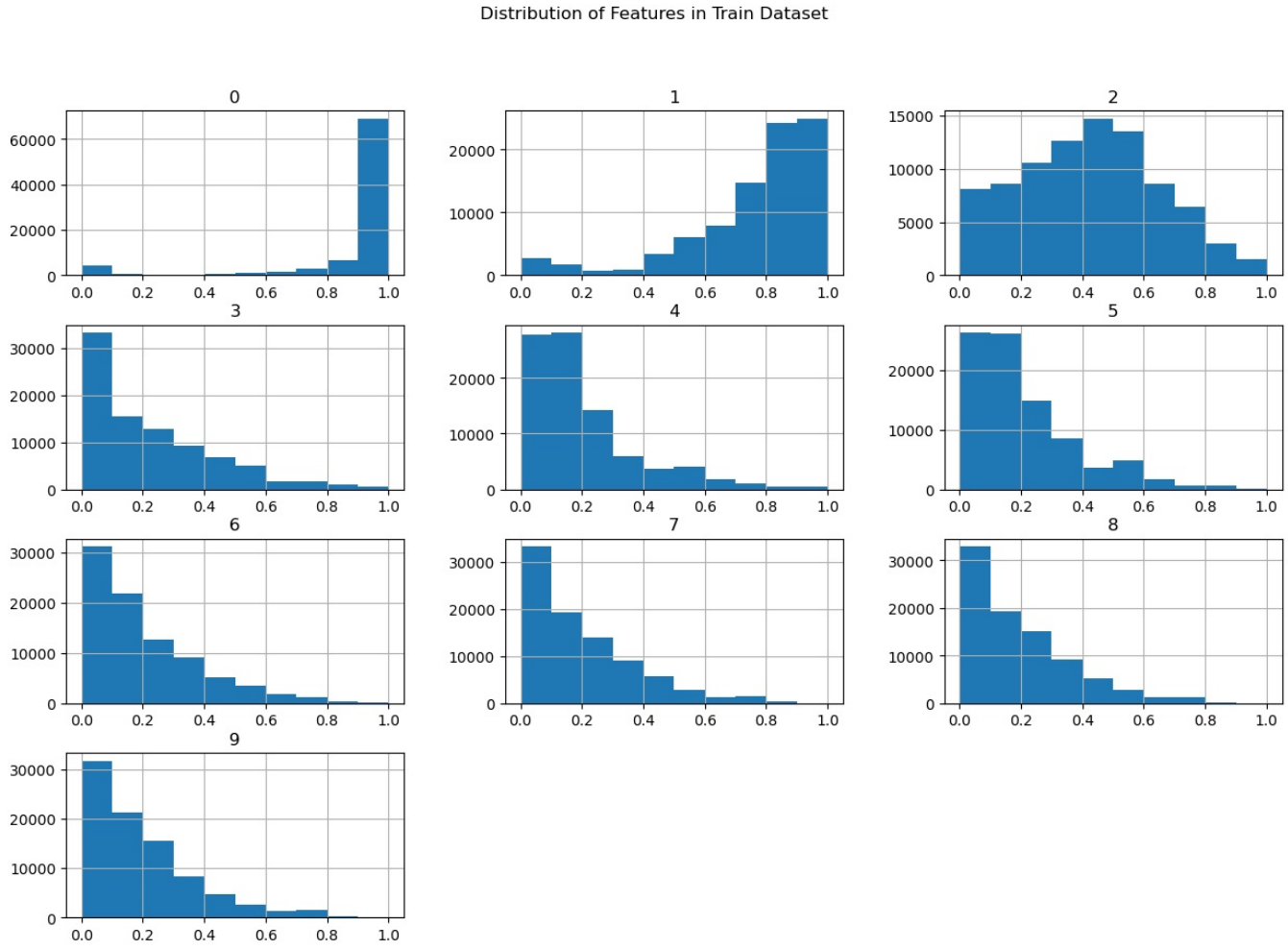
Test Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21892 entries, 0 to 21891
Columns: 188 entries, 0 to 187
dtypes: float64(188)
memory usage: 31.4 MB
None
```

In [6]:
```
print("\nMissing values in Train Dataset:")
print(train.isnull().sum())
print("\nMissing values in Test Dataset:")
print(test.isnull().sum())
```

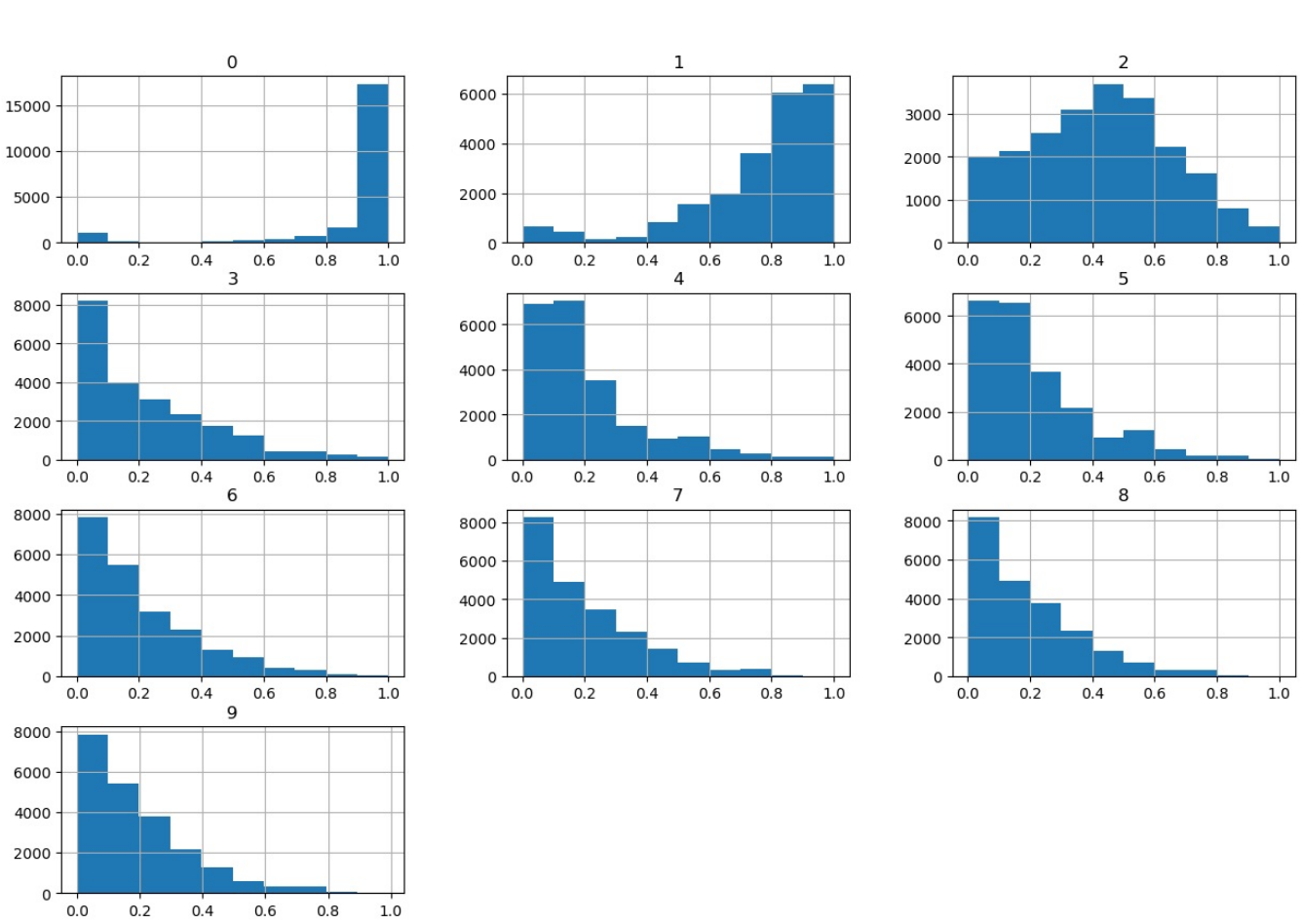
```
Missing values in Train Dataset:
0      0
1      0
2      0
3      0
4      0
      ..
183    0
184    0
185    0
186    0
187    0
Length: 188, dtype: int64

Missing values in Test Dataset:
0      0
1      0
2      0
3      0
4      0
      ..
183    0
184    0
185    0
186    0
187    0
Length: 188, dtype: int64
```

In [8]:
```
features_sample = train.columns[:10]  # Taking first 10 features as a sample
train[features_sample].hist(figsize=(15, 10))
plt.suptitle("Distribution of Features in Train Dataset")
plt.show()
```



In [9]:
```
features_sample = test.columns[:10]
test[features_sample].hist(figsize=(15, 10))
plt.suptitle("Distribution of Features in Test Dataset")
plt.show()
```



In [10]:
```
train.loc[:,187].astype('int').value_counts()
```

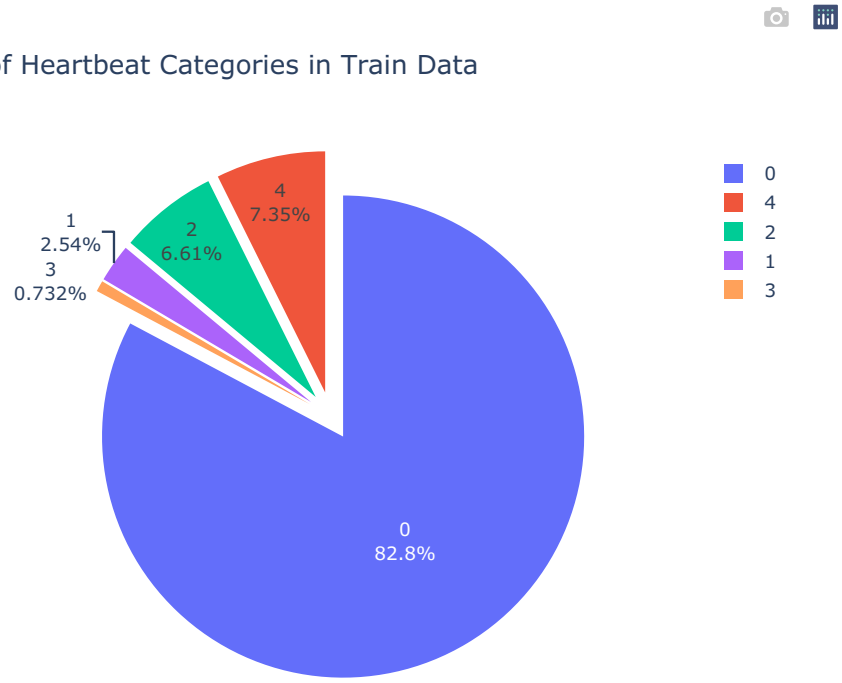
Out[10]:
```
187
0    72471
4     6431
2     5788
1     2223
3      641
Name: count, dtype: int64
```

In [11]:
```
import plotly.graph_objs as go

# Get the value counts for the target variable
value_counts = train[187].value_counts()

# Create the 3D pie chart
labels = value_counts.index
values = value_counts.values

fig = go.Figure(data=[go.Pie(labels=labels, values=values, pull=[0.1, 0.1, 0.1, 0.1, 0.1])]) # 'pull' gives a 3
fig.update_traces(textinfo='percent+label')
fig.update_layout(title_text="Distribution of Heartbeat Categories in Train Data")
fig.show()
```



The above pie chart clearly illustrates the data imbalance. More than 80% of the data belongs to class 0. To enhance prediction precision, we first need to balance the data. Let's apply undersampling to reduce the instances of class 0, making it more comparable to the other classes.

In [12]:
```
class_0 = train[train[187]==0.0].sample(n = 8000)
class_1 = train[train[187]==1.0]
class_2 = train[train[187]==2.0]
class_3 = train[train[187]==3.0]
class_4 = train[train[187]==4.0]
```

In [13]:
```
mit = pd.concat([class_0, class_1, class_2, class_3, class_4])
mit.head()
```

Out[13]:

	0	1	2	3	4	5	6	7	8	9	...	178	179	180	181	182	183	184
12076	1.0	0.773292	0.605590	0.394410	0.291925	0.161491	0.049689	0.018634	0.000000	0.006211	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
62456	1.0	0.829508	0.609836	0.327869	0.160656	0.154098	0.193443	0.252459	0.262295	0.262295	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
53956	1.0	0.867830	0.638404	0.309227	0.099751	0.069825	0.037406	0.022444	0.017456	0.000000	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
14743	0.0	0.115493	0.239437	0.380282	0.459155	0.563380	0.681690	0.729577	0.732394	0.743662	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
57293	1.0	0.834328	0.447761	0.082090	0.000000	0.094030	0.094030	0.062687	0.050746	0.044776	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 188 columns

In [34]:
```
import matplotlib.pyplot as plt

# Get the value counts for the target variable in the 'mit' DataFrame
value_counts_mit = mit[187].value_counts()

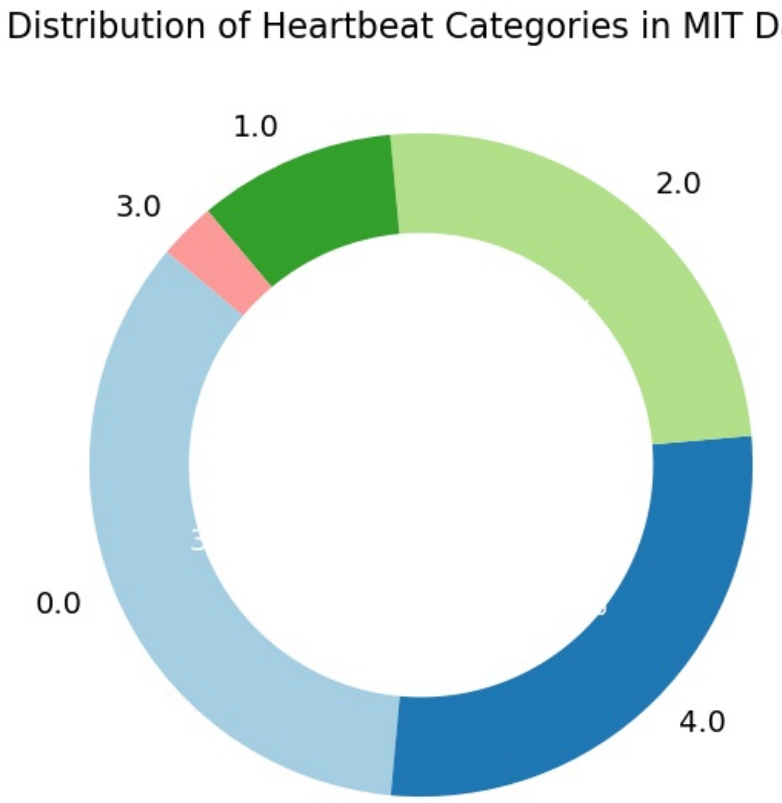
# Data for the doughnut chart
labels_mit = value_counts_mit.index
values_mit = value_counts_mit.values

# Colors for the categories
colors = plt.cm.Paired(range(len(labels_mit)))

# Plotting the doughnut chart
plt.figure(figsize=(10, 7))
wedges, texts, autotexts = plt.pie(values_mit, labels=labels_mit, colors=colors,
                                   autopct='%1.1f%%', startangle=140,
                                   wedgeprops=dict(width=0.3))
plt.title("Distribution of Heartbeat Categories in MIT Data", fontsize=16)

# Improve appearance of texts and autotexts
for text in texts:
    text.set(size=14, color='black')
for autotext in autotexts:
    autotext.set(size=14, color='white')
```

```
plt.show()
```



In [15]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(mit.drop([187], axis=1), mit[187], test_size = 0.2)
```

In [16]:
```
X_train = np.array(X_train).reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = np.array(X_test).reshape(X_test.shape[0], X_test.shape[1], 1)
```

MODEL BUILDING

In [17]:
```
import tensorflow as tf
from tensorflow.keras import Sequential,utils
from tensorflow.keras.layers import Flatten, Dense, Conv1D, MaxPool1D, Dropout
```

In [18]:
```
clf = tf.keras.Sequential([
    Conv1D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=(X_train.shape[1],1)),
    Conv1D(filters=64, kernel_size=3, padding='same', activation='relu'),
    Conv1D(filters=128, kernel_size=5, padding='same', activation='relu'),
    MaxPool1D(pool_size=3, strides=2, padding='same'),
    Dropout(0.5),
    Flatten(),
    Dense(units=512, activation='relu'),
    Dense(units=1024, activation='relu'),
    Dense(units=5, activation='softmax')
])
```

Framework: TensorFlow's Keras API

Model Architecture:

- **Conv1D Layer**:
  - Applies 1D convolution on the input data.
  - Uses 32 filters, each of size 3.
  - Activation function: 'ReLU'.
  - Padding: 'same' (output has the same width and height as the original input).
  - Purpose: Initial feature extraction from the ECG data.

- **Conv1D Layer**:
  - Applies 1D convolution with 64 filters, kernel size of 3.
  - Activation function: 'ReLU'.

- Purpose: Extracts more intricate features from the data.

- **Conv1D Layer**:
  - Applies 1D convolution with 128 filters, kernel size of 5.
  - Activation function: 'ReLU'.
  - Purpose: Recognizes more complex patterns in the data.

- **MaxPool1D Layer**:
  - Performs max pooling with a pool size of 3 and a stride of 2.
  - Purpose: Down-samples the feature maps while retaining crucial features.

- **Dropout Layer**:
  - Drops 50% of the neurons randomly during training.
  - Purpose: Prevents overfitting.

- **Flatten Layer**:
  - Converts 2D feature maps into a 1D vector.
  - Purpose: Prepares data for input into the dense layers.

- **Dense Layer**:
  - Fully connected layer with 512 neurons.
  - Activation function: 'ReLU'.

- **Dense Layer**:
  - Fully connected layer with 1024 neurons.
  - Activation function: 'ReLU'.

- **Dense Layer**:
  - Output layer with 5 neurons corresponding to the 5 ECG signal classes.
  - Activation function: 'softmax' (outputs a probability distribution over the 5 classes).

Objective: Classify ECG signals into one of the 5 categories, aiming for high accuracy and precision.

COMPILE MODEL

In [19]:
```
clf.compile(optimizer='adam', loss = 'sparse_categorical_crossentropy', metrics=['accuracy'])
```

- **Optimizer**: `adam`
  - A popular and efficient optimization algorithm that adjusts model weights to minimize the loss function.

- **Loss Function**: `sparse_categorical_crossentropy`
  - Suitable for classification tasks with multiple classes.
  - "Sparse" indicates that the labels are provided as integers rather than one-hot encoded vectors.

- **Metrics**: `accuracy`
  - Measures the proportion of correctly classified instances out of the total instances.

The `compile` method configures the model for training, setting the optimizer, loss function, and metrics that will be evaluated during the training process.

In [20]:
```
history = clf.fit(X_train, y_train, epochs = 20,
                  callbacks=[tf.keras.callbacks.EarlyStopping(monitor='loss', patience=5, restore_best_weights=
```

```
Epoch 1/20
578/578 [==============================] - 110s 184ms/step - loss: 0.4059 - accuracy: 0.8581
Epoch 2/20
578/578 [==============================] - 106s 184ms/step - loss: 0.2143 - accuracy: 0.9273
Epoch 3/20
578/578 [==============================] - 106s 183ms/step - loss: 0.1607 - accuracy: 0.9469
Epoch 4/20
578/578 [==============================] - 143s 248ms/step - loss: 0.1306 - accuracy: 0.9565
Epoch 5/20
578/578 [==============================] - 149s 258ms/step - loss: 0.1063 - accuracy: 0.9627
Epoch 6/20
578/578 [==============================] - 113s 195ms/step - loss: 0.0910 - accuracy: 0.9682
Epoch 7/20
578/578 [==============================] - 115s 199ms/step - loss: 0.0782 - accuracy: 0.9724
Epoch 8/20
578/578 [==============================] - 111s 193ms/step - loss: 0.0683 - accuracy: 0.9747
Epoch 9/20
578/578 [==============================] - 111s 193ms/step - loss: 0.0679 - accuracy: 0.9763
Epoch 10/20
578/578 [==============================] - 129s 223ms/step - loss: 0.0547 - accuracy: 0.9789
Epoch 11/20
578/578 [==============================] - 116s 200ms/step - loss: 0.0469 - accuracy: 0.9835
Epoch 12/20
578/578 [==============================] - 117s 202ms/step - loss: 0.0456 - accuracy: 0.9820
Epoch 13/20
578/578 [==============================] - 122s 211ms/step - loss: 0.0463 - accuracy: 0.9845
Epoch 14/20
578/578 [==============================] - 125s 216ms/step - loss: 0.0366 - accuracy: 0.9871
Epoch 15/20
578/578 [==============================] - 132s 229ms/step - loss: 0.0306 - accuracy: 0.9891
Epoch 16/20
578/578 [==============================] - 120s 208ms/step - loss: 0.0385 - accuracy: 0.9868
Epoch 17/20
578/578 [==============================] - 128s 222ms/step - loss: 0.0330 - accuracy: 0.9883
Epoch 18/20
578/578 [==============================] - 120s 207ms/step - loss: 0.0329 - accuracy: 0.9888
Epoch 19/20
578/578 [==============================] - 120s 207ms/step - loss: 0.0285 - accuracy: 0.9894
Epoch 20/20
578/578 [==============================] - 130s 225ms/step - loss: 0.0268 - accuracy: 0.9898
```

MAKE PREDICTION

In [21]:
```python
y_pred = clf.predict(X_test)
```

```
145/145 [==============================] - 4s 27ms/step
```

In [22]:
```python
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Extract accuracy and epochs from the history object
acc = history.history['accuracy']
epochs = list(range(1, len(acc) + 1))

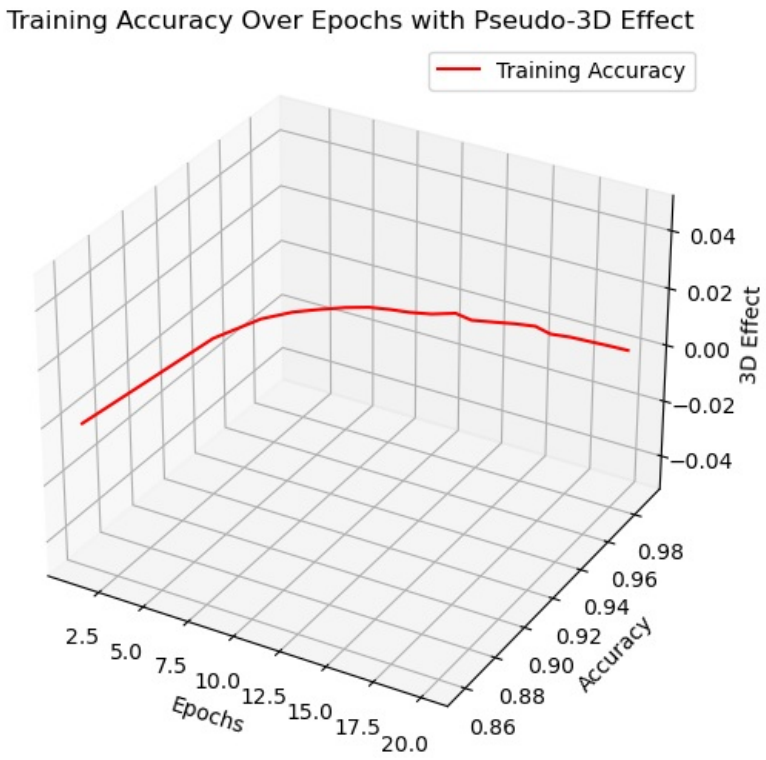
# Create a new figure
fig = plt.figure(figsize=(10, 6))
ax = fig.add_subplot(111, projection='3d')

# Plot the data: the z-values are all zero to keep the line on the "floor"
ax.plot(epochs, acc, zs=0, zdir='z', c='r', label='Training Accuracy')

# Set labels and title
ax.set_xlabel('Epochs')
ax.set_ylabel('Accuracy')
ax.set_zlabel('3D Effect')  # Just a dummy axis for 3D effect
ax.set_title('Training Accuracy Over Epochs with Pseudo-3D Effect')
ax.legend()

# Display the plot
plt.show()

# Save the figure
fig.savefig("train-accuracy-3d-matplotlib.jpg")
```



In [23]:
```
from sklearn import metrics
y_lbl = [np.where(i == np.max(i))[0][0] for i in y_pred]
print("Train Accuracy: {0:.3f}".format(metrics.accuracy_score(y_lbl, y_test)))
```

```
Train Accuracy: 0.956
```

In [38]:
```
import matplotlib.pyplot as plt

# Get the value counts for the target variable in the 'test' DataFrame
value_counts_test = test[187].value_counts().sort_index()

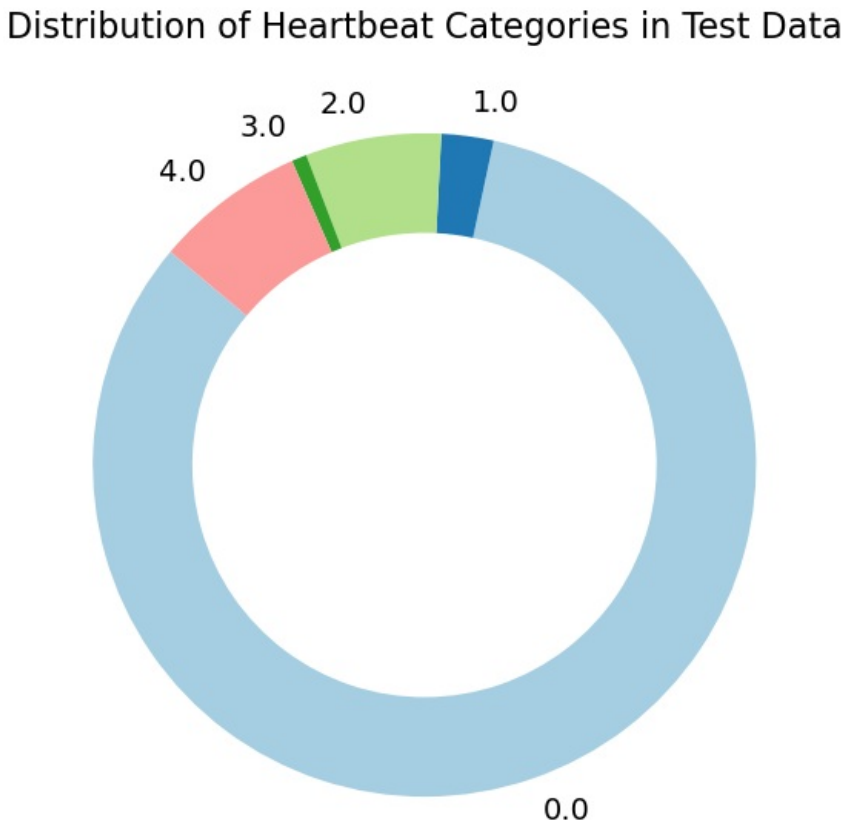
# Data for the doughnut chart
labels_test = value_counts_test.index
values_test = value_counts_test.values

# Colors for the categories
colors = plt.cm.Paired(range(len(labels_test)))

# Plotting the doughnut chart
plt.figure(figsize=(10, 7))
wedges, texts, autotexts = plt.pie(values_test, labels=labels_test, colors=colors,
                                   autopct='%1.1f%%', startangle=140,
                                   wedgeprops=dict(width=0.3))

# Enhancing the appearance of texts and autotexts
for text in texts:
    text.set(size=14, color='black')
for autotext in autotexts:
    autotext.set(size=14, color='white')

plt.title("Distribution of Heartbeat Categories in Test Data", fontsize=16)
plt.show()
```



In [25]:
```
test_X = test.drop([187], axis=1)
test_y = test[187]

test_X = np.array(test_X).reshape(test_X.shape[0], test_X.shape[1], 1)
```

In [26]:
```
test_pred_y = clf.predict(test_X)
```

```
685/685 [==============================] - 19s 27ms/step
```

In [27]:
```
test_lbl_y = [np.where(i == np.max(i))[0][0] for i in test_pred_y]
print("Test Accuracy: {0:.3f}".format(metrics.accuracy_score(test_lbl_y, test_y)))
```

```
Test Accuracy: 0.964
```

In [28]:
```
from sklearn.metrics import classification_report
print(classification_report(test_lbl_y, test_y))
```

```
              precision    recall  f1-score   support

           0       0.97      0.99      0.98     17609
           1       0.85      0.56      0.68       849
           2       0.96      0.90      0.93      1557
           3       0.82      0.63      0.71       211
           4       0.99      0.96      0.98      1666

    accuracy                           0.96     21892
   macro avg       0.92      0.81      0.85     21892
weighted avg       0.96      0.96      0.96     21892
```

In [29]:
```
report = classification_report(test_lbl_y, test_y, output_dict=True)
```

In [30]:
```
from mpl_toolkits.mplot3d import Axes3D

# Extract data from the report
classes = list(report.keys())[:-3]  # Get the class labels
metrics = ['precision', 'recall', 'f1-score']
values = [[report[cls][metric] for cls in classes] for metric in metrics]
accuracy = report['accuracy']

# Create 3D bar chart
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

colors = ['r', 'g', 'b']  # Assigning colors for each metric

# Defining the width and depth of the bars
dx = 0.5
dy = 0.5

# For each metric, plot its bars in the assigned color
```

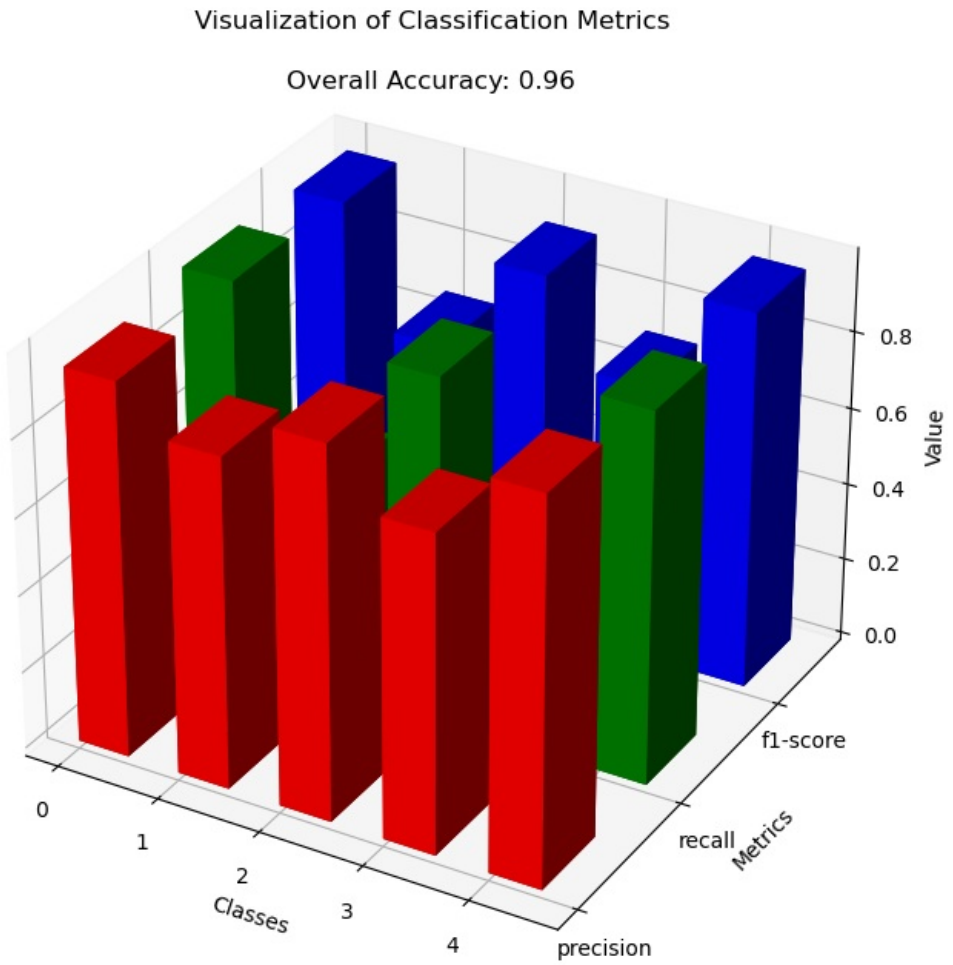
```
for metric_index, metric in enumerate(metrics):
    xpos = [i for i in range(len(classes))]
    ypos = [metric_index] * len(classes)
    zpos = [0] * len(classes)
    dz = values[metric_index]
    ax.bar3d(xpos, ypos, zpos, dx, dy, dz, color=colors[metric_index], shade=True)

ax.set_xticks([i for i in range(len(classes))])
ax.set_xticklabels(classes)
ax.set_yticks([i for i in range(len(metrics))])
ax.set_yticklabels(metrics)

ax.set_xlabel('Classes')
ax.set_ylabel('Metrics')
ax.set_zlabel('Value')
ax.set_title(' Visualization of Classification Metrics')

# Adjusting the position of the accuracy text to be just below the title
ax.text2D(0.35, 0.95, f"Overall Accuracy: {accuracy:.2f}", transform=ax.transAxes, fontsize=12, color='black')

plt.show()
```



Classification Report:

	Precision	Recall	F1-Score	Support
Class 0	0.97	0.99	0.98	17609
Class 1	0.85	0.56	0.68	849
Class 2	0.96	0.90	0.93	1557
Class 3	0.82	0.63	0.71	211
Class 4	0.99	0.96	0.98	1666

Overall Metrics:

- **Accuracy**: 0.96 (21,892 instances)
- **Macro Average**: Precision: 0.92, Recall: 0.81, F1-Score: 0.85
- **Weighted Average**: Precision: 0.96, Recall: 0.96, F1-Score: 0.96
